Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Bağlam farkında sinirsel dil modelleri: neden gerekli?
  - Daha önce görülen anlamsal uzaylarda (word2vec vb.) her kelimeye tek bir vektör atanıyordu. Çok anlamlı kelimelerde bu bir sorundur: "çay içiyorum" ile "akan çayın yanına indim" cümlelerindeki "çay" aynı vektörü alır.
  - Çözüm: kelimenin vektörünün etrafındaki kelimelere (bağlama) göre değişebilmesi. Dersin planı (ekrandan, slayt "İçerik"): dağıtık temsil, RNN ve türevleri (LSTM, GRU, attention, transformer), self-supervised learning, byte pair encoding, bağlam tabanlı dil modelleri ve BERT'in kullanımı.
- Dağıtık temsil (Distributed representations) (ekrandan, 00:07:40 ve 00:10:00)
  - Resimdeki insanları kadın/erkek, gözlüklü/gözlüksüz, uzun/kısa diye ayırmak için 8 sınıf tanımlanırsa her sınıftan örnek gerekir. 3 alt özelliği öğrenen 3 ikili (binary) sınıflandırıcı tanımlanırsa eğitim kümesinde "uzun, kadın, gözlüksüz" örneği olmasa bile bu kombinasyon tanınabilir.
  - d alt özellikle 2^d farklı kombinasyon ifade edilir. N sınıf örtüşmeli (non mutually exclusive) özellikler taşıyorsa $\log_2 N$ alt özellikle ifade edilebilir.
  - Parametre sayısı: 1024 sınıf ve $d \times d$'lik resimler için klasik ağ (giriş doğrudan çıkışa) $(d \cdot d) \cdot 1024$ parametre öğrenir. Araya 10 alt özellikli bir katman eklenirse $(d \cdot d) \cdot 10 + 10 \cdot 1024$ parametre öğrenilir. Son katman alt özelliklerin kombinasyonlarını sınıf etiketlerine eşlemeyi öğrenir. Sonuç: katman sayısını artırarak öğrenilecek parametre sayısı azaltıldı.
  - Ders sonrası not: slaytta $d = 128$ için klasik yaklaşım "yaklaşık 4 milyon (2^{24})" yazıyor. $128 \cdot 128 \cdot 1024 = 2^{24} = 16\,777\,216$, yani yaklaşık 16,8 milyondur (python ile hesaplandı). Dağıtık yaklaşım $128 \cdot 128 \cdot 10 + 10 \cdot 1024 = 174\,080$, slayttaki "yaklaşık 180 bin" ile uyumludur.
  - Alt özellikler önceden bilinmez, ağın kendisi öğrenir (ekrandan, 00:20:00): MLP'de her gizli nöron, CNN'de her filtre bir alt özellik öğrenir. Varsayım: tüm resimler ortak küçük parçalardan oluşur. Yüz, araba, fil, sandalye sınıflarının hepsi en alt katmanda aynı basit özellikleri (çizgiler, koyu/açık alanlar) içerir; üst katmanlar bunların kombinasyonlarını öğrenir.
  - CNN'de pooling bir alt özelliğin yalnız var olup olmadığına bakar, yerine bakmaz. Bir yüzün parçaları yer değiştirse de CNN yüz diyebilir. Hinton'ın kapsül ağları (capsule networks) parçaların birbirine göre konumlarını da işe katar.
- Tekrarlayan sinir ağları (RNN) (ekrandan, 00:30:00)
  - Girişin bir sekans olduğu uygulamalar için: cümlenin kelimeleri, filmin kareleri, konuşmanın fonemleri. Sıra karışırsa anlam kaybolur. Döviz kuru örneği: yalnız dünkü değere bakarak bugünkü tahmin edilemez, geçmişin gidişatı da gerekir.
  - h_t: t anındaki çıkış, x_t: t anındaki giriş. h_t, x_t ve h_{t-1}'e bağlıdır; h_{t-1} başlangıçtan o ana kadarki bütün geçmişi sıkıştırılmış olarak taşır.
  - Klasik ağlar da girişe sabit büyüklükte bir geçmiş $(x_t, x_{t-1}, \ldots, x_{t-k})$ verilerek zamanı kullanabilir. CNN'ler de bunu yapabilir: kelime vektörleri (ör. 300 boyutlu) yan yana dizilince cümle bir matrise (resme) dönüşür; 3×300'lük bir filtre kaydırıldığında ardışık üç kelimeye birlikte bakılır.
  - Uygulama türleri (ekrandan, 00:40:00): one to one (klasik ağ), one to many (resim girer, onu anlatan cümle çıkar), many to one (tweet girer, duygu çıkar), many to many (cümle girer, başka dildeki karşılığı çıkar; giriş ve çıkış uzunlukları farklı olabilir).

  - Temel denklem: $h_t = f(W^{hh}h_{t-1} + W^{hx}x_t)$. W^{hx} girişin durumu nasıl etkilediğini, W^{hh} önceki durumdan sonraki duruma geçişi öğrenir (Markov yapılarındaki geçiş matrisine benzer). Çıkış için ayrıca W^{hy} öğrenilir; bütün iş bu üç matrisi öğrenmektir.
  - Bir sonraki harfi tahmin örneği (ekrandan, 00:44:40): 4 karakterlik sözlük (h, e, l, o), one-hot giriş ve çıkış, 3 gizli nöron; "hell" girişleri için hedef karakterler "e, l, l, o".
- RNN'lerle dil çevirisi (ekrandan, 00:50:00)
  - Encoder-decoder (diziden diziye) yapısı. Encoder: $h_t = f(W^{hh}h_{t-1} + W^{hx}x_t)$; kaynak cümlenin kelimeleri tek tek verilir ve bütün cümle son durum vektörüne sıkıştırılır. Decoder: $h_t = f(W^{hh}h_{t-1})$, $y_t = \text{softmax}(W^S h_t)$; artık giriş yoktur, her kelime üretildikten sonra durum güncellenir. Slayttaki örnek: Almanca "Echt dicke Kiste" → İngilizce "Awesome sauce".
  - Sorun: bütün cümleyi tek bir vektöre indirip ondan yeni bir dizi üretmek zordur; bu mimari kural tabanlı çevirinin çok ötesine geçememiştir.
  - İyileştirmeler (ekrandan, 01:20:00): encoder'a cümleyi tersten vermek (ABC → DEF yerine CBA → DEF) eşlenecek kelimeler arasındaki mesafeyi kısaltır. Decoder'da $h_t = \varphi(h_{t-1}, C, y_{t-1})$: önceki çıktı y_{t-1} ve bağlam vektörü C her adımda kullanılır; her biri için ayrı bir W vardır. Çıkış $y_t = \text{softmax}(W^S[h_t; C])$.
- LSTM ve GRU (ekrandan, 01:15:30 ve 01:17:10)
  - Çıkış noktası: her giriş h_t'yi etkilemek zorunda değildir (anlamı az değiştiren kelimeler), bazen de bağlamın sıfırlanması gerekir (bir cümle bitip ikincisine geçerken). LSTM bu iki amaçla geliştirildi (1990'lar).
  - LSTM'de h_t'nin yanında ikinci bir bağlam yolu, hücre durumu C_t ilerler. Denklemler: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$, $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$, $\tilde{C_t} = \tanh(W_C[h_{t-1}, x_t] + b_C)$, $C_t = f_t * C_{t-1} + i_t * \tilde{C_t}$, $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$, $h_t = o_t * \tanh(C_t)$. C_{t-1} belli oranda unutulur ya da saklanır; dört W matrisi öğrenilir.
  - GRU daha basittir, 3 matris (W_z, W_r, W) öğrenir: $z_t = \sigma(W_z[h_{t-1}, x_t])$, $r_t = \sigma(W_r[h_{t-1}, x_t])$, $\tilde{h}_t = \tanh(W[r_t * h_{t-1}, x_t])$, $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$. LSTM ile benzer başarıdadır; bunun teorik bir açıklaması yoktur, deneysel olarak görülmüştür.
- Attention (dikkat) (ekrandan, 01:21:30)
  - Encoder'ın tüm durumları etkindir, ama ne ölçüde etkin olacakları (α) öğrenilir. $\alpha^{i,j}$: i. çıkışa j. girişin ne kadar etkisi olacağı; $\sum_t \alpha^{1,t} = 1$; $h^t = (\vec{h}^t, \overleftarrow{h}^t)$ (çift yönlü LSTM); $C_1 = \sum_t \alpha^{1,t} h^t$.
  - 20 kelimelik giriş ve 20 kelimelik çıkış için $20 \cdot 20 = 400$ katsayı öğrenilir.
  - "Attention is all you need" makalesiyle tekrarlayan yapı tamamen çıkarıldı; yalnız attention'a dayanan Transformer'lar makine çevirisinde en iyi sonuçlara ulaştı.
  - Self-attention: bir cümledeki her kelimenin aynı cümledeki diğer kelimelere ne kadar bağlı olduğu. "Ali okula gelince onu gören Hilmi çok sevindi" cümlesinde "onu" kelimesinin vektörü "Ali"ye, "Kedi okula gelince …" cümlesinde "kedi"ye yakın olmalıdır. word2vec'te "onu" her cümlede aynı vektörü alır. Transformer hem girişin hem çıkışın kendi içindeki bağımlılıklarına bakar.
- Bağlam tabanlı dil modelleri (ekrandan, 01:30:00)
  - RNN tabanlı: TagLM, önceden eğitilmiş (sonraki kelimeyi tahmin için eğitilmiş) bir modelin başka bir görevde ilk kullanımı. ELMo: çok sayıda görevde performans artışı; üzerine küçük bir sınıflandırma katmanı eklenip göreve göre ince ayar (fine-tune) yapılır.
  - Transformer tabanlı: GPT (sonraki kelimeyi tahmin), BERT (maskelenmiş kelimeyi tahmin + sonraki cümle mi ikili sınıflandırması; pozitif: ardışık iki cümle, negatif: rastgele iki cümle), RoBERTa (yalnız maskelenmiş kelime tahmini), ALBERT (negatif örnek olarak pozitifteki iki cümlenin yeri değiştirilir, daha zor bir soru), ELECTRA (değiştirilmiş kelimeleri tahmin), BART (çeşitli görevler).
  - word2vec'te olduğu gibi bu ön eğitim görevleri insan etiketlemesi gerektirmez; derlemden cümle seçilip kelimeler kapatılır, doğru cevap zaten bilinir.
- Self-supervised görevler (ekrandan, 01:40:00)
  - Büyük miktarda eğitim verisi üretmek için kurulan yan görevler; insan emeği gerekmez. Sonraki kelime, cümle ya da film karesini tahmin; gürültü ekleme: maskelen-

miş kelime tahmini, sırası bozulmuş cümleyi düzeltme, harfleri bozulmuş kelimeyi düzeltme, resmin silinmiş kısmını tahmin.
  - BART görevleri (orijinal cümle "ABC.DE."): Token Masking "A_C._E.", Token Deletion "A.C.E.", Text Infilling "A_.D_E.", Sentence Permutation "DE.ABC.", Document Rotation "C.DE.AB".
  - Görüntüde: karşıtlıklı (contrastive) öğrenmede aynı resimden alınan parçaların benzer, farklı resimlerden alınanların farklı olduğu öğrenilir; resmi 4×4'lük parçalara bölüp karıştırma ve orijinaline döndürme.
- Byte pair encoding (BPE) (ekrandan, 01:42:20 ve 01:50:00)
  - Veri sıkıştırmak için önerilmiş bir yöntem; en sık geçen ardışık ikili yeni bir sembolle değiştirilir: aaabdaaabac → ZabdZabac (Z=aa) → ZYdZYac (Y=ab) → XdXac (X=ZY).
  - Sinirsel dil modellerinde çok yaygındır; amaç bilinmeyen (unknown, OOV) token'lardan kurtulmaktır. En küçük birim karakter olduğundan sözlükte olmayan bir kelime de parçalara (en kötü durumda karakterlere) bölünür: "İlkbaharxdefsr" → "İlk + bahar + x + d + e + f + s + r" gibi.
  - "Neural Machine Translation of Rare Words with Subword Units" makalesiyle popüler oldu. Kütüphaneler: WordPiece, SentencePiece.
  - Türkçe gibi eklemeli dillerde yalnız OOV için değil normal kelimeler için de çok önemlidir: gel-ecekmiş, bil-ecekmiş, bit-miş, gül-müştü. Parçalar morfolojik analizden çıkmış gibi görünür, ama tamamen frekans tabanlıdır. Bir derlem verilip istenen büyüklükte (ör. 64 bin) sözlük ürettirilir. Böylece morfolojik analiz (Zemberek vb.) gereği büyük ölçüde ortadan kalktı.
- Büyük resim ve hazır modeller
  - ELMo, ULMFiT, GPT, BERT, GPT-2, GPT-3 ile parametre sayısında patlama. Transformer'lar görüntü işlemede CNN'lerin yerini almaya da başladı.
  - Türkçe için hazır modeller var (BERTurk, GPT-2'nin Türkçesi); metin sınıflandırma, eksik kelime tahmini gibi örnekler bulunuyor.
- BERT nasıl çalışır (hocanın "nasıl_calisir_BERT" notları, ekrandan, 02:10:00 - 02:43:15)
  - ELMo sola ve sağa giden LSTM'leri birleştirir; OpenAI GPT soldan sağa Transformer kullanır (her kelime yalnız öncekilere bakar); BERT çift yönlü Transformer'dır, her kelime her kelimeye bakar.
  - Self-attention: her kelime için key (K), value (V) ve query (Q) vektörleri, öğrenilen W_K, W_V, W_Q matrisleriyle çarpılarak (izdüşüm) üretilir. Bir kelimenin query'si ile tüm key'lerin iç çarpımı benzerlikleri verir (softmax'tan geçirilir); kelimenin yeni temsili, value'ların bu benzerliklerle ağırlıklı toplamıdır. Bu yüzden aynı kelime her cümlede farklı temsil alır.
  - Giriş gömmeleri: token embedding (BPE parçaları, "play ##ing" gibi) + segment embedding (A/B cümlesi) + position embedding. Transformer'da her şey tek seferde verildiği için konum bilgisi yoktur, model bag of words gibi kalır; konum gömmeleri bunu çözer. Hoca bunu farklı frekanslı sinüs fonksiyonlarıyla anlattı: yavaş sinüs cümlenin başında mı sonunda mı olunduğunu, hızlı sinüs yan yana kelimelerin sırasını ayırır.
  - Ön eğitim dizisi [CLS] ile başlar, cümleler [SEP] ile ayrılır. NSP için [CLS]'nin çıkış vektörü C bir W ile sınıflandırılır (IsNext/NotNext). MLM için her dizide token'ların %15'i rastgele maskelenir; maskeli konumların çıkışları $W \in \mathbb{R}^{V \times H}$ ile çarpılıp sözlük üzerinde softmax alınır: $\text{softmax}(CW^T) \in \mathbb{R}^{1 \times V}$. Toplam kayıp iki tahminin kayıplarının toplamıdır. Ön eğitimden sonra üstteki sınıflandırma katmanı atılır.
  - İnce ayar örnekleri: cümle çifti sınıflandırma (C vektörü × H×sınıf sayısı boyutlu W), soru cevaplama (SQuAD v1.1; soru ve paragraf verilir, Q_{start} ve Q_{end} vektörleri paragraf token'larının çıkışlarıyla iç çarpılır, en yüksek değer cevabın başı ve sonudur), adlandırılmış varlık tanıma (CoNLL-2003 NER; her token'ın çıkışı ortak bir W ile varlık türüne eşlenir), çoktan seçmeli sorular (SWAG; her şık için "soru + şık" girdisi, öğrenilen tek bir R vektörü ile her C'nin iç çarpımı, en büyüğü cevap).
  - Feature-based kullanım: BERT'i ince ayarlamadan temsillerini alıp başka bir modelde kullanmak; bu da iyi sonuç vermiştir. 12 katmanın her biri her token için bir çıkış üretir; NER üzerinde sondan bir önceki katman ve son 4 katmanın birleştirilmesi son katmandan daha iyi sonuç vermiştir (makalede tablo 7). Açıklama: son katman ön eğitim görevlerine (MLM, NSP) fazla odaklanmıştır.
  - Özetle BERT bir encoder'dır: dizi üretmez, temsil üretir. "Attention is all you

need”daki encoder-decoder yapısının yalnız encoder kısmını kullanır.

Hocanın Özellikle Vurguladığı Kısımlar

- Alt özellik yoksa dağıtık temsil olmaz
  - Bir öğrencinin “1024 sınıf hiç ortak parça içermiyorsa?” sorusuna hoca: bütünler parçalardan oluşmuyorsa öğrenilecek bir şey yoktur, sistem bir arama tablosuna (look up table) döner. Amaç aynı sınıftaki örneklerin benzer, farklı sınıflardakilerin farklı alt özelliklerini görmektir.
- RNN’lerin bütün hikayesi üç matris
  - W^{hx}, W^{hh} ve W^{hy} öğrenildiğinde sistemin modeli ortaya konmuş olur. LSTM’de 4, GRU’da 3 matris öğrenilir.
- Tek vektöre sıkıştırma sorunu
  - Klasik encoder-decoder’da bütün cümle tek bir vektöre indirilir; bunu öğrenmek kolay değildir. Attention ve Transformer’lar bu darboğazı aşmak için doğdu.
- Self-supervised görevlerin değeri
  - Başarılı ve yararlı görevler üretilebilirse etiketleme yapmadan “bedavadan” bilgi kazanılır; önceden eğitilmiş modellerin gücü buradan gelir.
- BPE’nin Türkçe için önemi
  - Eklemeli dillerde kelime sayısının patlamasını ve bilinmeyen kelime sorununu çözer; parçalar dil bilgisinden değil frekanstan gelir.
- İnce ayarın maliyeti
  - Cümle çifti sınıflandırmada öğrenilen ek parametre yalnız tek bir W matrisidir; BERT’in yüz milyonlarca parametresi yanında “devede kulak” kalır. Tüm ağ da güncellenebilir, bu tam ince ayardır.

Kısa Tekrar Notları

- Statik kelime vektörü çok anlamlılığı çözemez; bağlam farkında modeller kelimeye cümleye göre vektör verir.
- d alt özellik 2^d kombinasyon; N sınıf için $\log_2 N$ alt özellik yeter. Ara katman parametre sayısını azaltır.
- RNN: $h_t = f(W^{hh}h_{t-1} + W^{hx}x_t)$, çıkış W^{hy} ile.
- Uygulama türleri: one to one, one to many, many to one, many to many.
- Encoder-decoder: cümle tek vektöre (C) sıkıştırılır; iyileştirme: cümleyi tersten vermek, decoder’a C ve y_{t-1}’i her adımda vermek.
- LSTM: unutma, giriş, çıkış kapıları ve hücre durumu C_t; 4 matris. GRU: güncelleme ve sıfırlama kapıları; 3 matris.
- Attention: $C_i = \sum_j \alpha^{i,j} h^j$, $\sum_j \alpha^{i,j} = 1$.
- Self-attention: Q, K, V; query × key iç çarpımı → softmax → value’ların ağırlıklı toplamı.
- Ön eğitim görevleri: GPT sonraki kelime, BERT MLM + NSP, RoBERTa yalnız MLM, ALBERT cümle sırası, ELECTRA değiştirilmiş kelime, BART gürültü giderme.
- BPE: en sık ikiliyi birleştir; OOV olmaz, Türkçe ekler kendiliğinden ayrışır.
- BERT girişi: token + segment + position embedding; [CLS], [SEP].
- BERT ince ayarı: sınıflandırma için C + W; soru cevaplama için Q_{start}, Q_{end}; NER için token başına W; çoktan seçmeli için R vektörü.
- Feature-based BERT: temsilleri al, başka modelde kullan; sondan bir önceki ya da son 4 katman daha iyi.

Detaylı Açıklamalar

- Dağıtık temsilin parametre kazancı
  - Klasik yaklaşımda girişten çıkışa tek bir W öğrenilir ve boyutu (giriş boyutu) × (sınıf sayısı) olur. Araya k boyutlu bir katman koymak bu matrisi iki küçük matrise böler: (giriş boyutu) × k ve k × (sınıf sayısı). k küçükse toplam parametre çok azalır. Bunun anlamlı olması için sınıfların ortak alt özelliklerden kurulabilmesi gerekir. Hoca renk örneğiyle açıkladı: 1024 farklı renk sınıfı, kırmızı, yeşil ve mavi gibi birkaç temel rengin farklı oranlarda birleşimiyle ifade edilebilir; bu temel renkler orta katmanda yer alır.

  - Ders sonrası not: $d = 128$ için slayttaki "yaklaşık 4 milyon (2^{24})" ifadesinde sayı ile üs çelişiyor; $2^{24} = 16\,777\,216 \approx 16{,}8$ milyon doğrudur.
- CNN ile cümle işleme ve öğrenci sorusu
  - Kelime vektörleri sütun sütun dizildiğinde satırlar aynı boyutları (1. satır bütün kelimelerin 1. boyutu) gösterir. Bir öğrenci, piksel gibi dikey komşulukların anlamsız ilişki kuracağını sordu. Hoca: 2×2 gibi küçük çekirdekler kaydırılsaydı bu doğru olurdu, ama burada çekirdek kelime boyutunun tamamını kapsar (3×300); yalnız kelime ekseninde kaydırılır, olmayan bir ilişki eklenmez.
- Encoder-decoder'da decoder'ın çalışması
  - Bir öğrenci ikinci kelimenin nasıl üretildiğini sordu. Hoca: ilk kelime ("Awesome") h_t üzerinden W^S ile üretilir; sonra h_t, W^{hh} ile güncellenir ve yeni durumdan ikinci kelime ("sauce") üretilir. Temel yapıda üretilen kelime bir sonraki adıma verilmez; bu, iyileştirilmiş yapıda (y_{t-1}'in eklenmesiyle) yapılır.
- Self-attention'ın sezgisi
  - "Onu" gibi bir zamir bağlamsız bir vektörle temsil edilirse göndermede bulunduğu varlığa ait bilgi taşımaz. Self-attention'da "onu"nun query'si diğer kelimelerin key'leriyle karşılaştırılır, en çok benzediği kelimenin value'su yeni temsile en çok katkıyı yapar. Böylece aynı kelime farklı cümlelerde farklı vektör alır. Hocanın notundaki örnek: a, b, c, d, e kelimelerinden q en çok b'ye benziyorsa yeni temsil ağırlıklı olarak b'nin value'sundan gelir (ekrandan, 02:10:00).
  - Hoca orijinal Transformer'da 512 boyutlu temsillerin key, value ve query için 64 boyuta indirildiğini, çok başlı (multi-head) yapıyla tekrar 512'ye çıkarıldığını belirtti.
- Konum gömmesi sezgisi
  - Farklı frekanslı sinüsler, konumu farklı çözünürlüklerde kodlar: düşük frekanslı sinüste birbirine yakın kelimelerin değerleri yakın, uzakların uzaktır; yüksek frekanslıda yan yana iki kelime bile ayrılır.
  - Ders sonrası not: sinüs tabanlı konum kodlaması orijinal Transformer makalesindeki yöntemdir; BERT'te konum gömmeleri büyük olasılıkla eğitimle öğrenilen vektörlerdir.
- BERT'in boyutları
  - Hoca örneklerde C vektörünü 512 boyutlu aldı; BERT-Large için yaklaşık 300 milyon, BERT-Base için 110 milyon parametre, sözlük için yaklaşık 30 bin token dedi.
  - Ders sonrası not: BERT-Base'in gizli boyutu büyük olasılıkla 768, BERT-Large'ınki 1024'tür; 512, BERT'in kabul ettiği en uzun giriş dizisinin token sayısıdır. Ayrıca GPT-3 için hoca "trilyona yaklaştı herhalde" dedi; GPT-3 büyük olasılıkla 175 milyar parametrelidir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.